

SCD1 / SCD2 Yelp Notebook — Line-by-Line Explained

Every cell and line of the SCD practice notebook, explained in detail

Delta · MERGE · SCD1 overwrite · SCD2 versioning · mergeKey pattern · validation

A detailed walkthrough of `scd1_scd2_yelp_practice.py`. For each cell, the code is explained line by line so you understand *what* it does and *why*. Concepts (Delta, MERGE, the SCD2 "mergeKey" pattern) are explained where they appear.

Cell: imports & setup

```
from pyspark.sql import functions as F
from delta.tables import DeltaTable
```

- **functions as F** — Spark's built-in column functions (`F.lit`, `F.col`, `F.sha2`, ...). Aliased `F` by convention so you write `F.lit(...)`.
- **DeltaTable** — the Delta Lake API object that lets you run `MERGE` (upsert) on a Delta table.

```
catalog = spark.catalog.currentCatalog()
```

- Gets the **current Unity Catalog catalog** name (e.g., `workspace`). Needed to build the Volume path, because UC paths are `/Volumes/<catalog>/<schema>/<volume>`.

```
spark.sql("CREATE SCHEMA IF NOT EXISTS yelp")
```

- Creates a **schema** (a.k.a. database) named `yelp` in the current catalog to hold our tables. `IF NOT EXISTS` makes it safe to re-run.

```
spark.sql("CREATE VOLUME IF NOT EXISTS yelp.files")
```

- Creates a **Volume** — a Unity Catalog object for storing *files* (Free Edition is serverless, so you store files in Volumes, not arbitrary DBFS paths). This is where our Parquet will live.

```
VOL = f"/Volumes/{catalog}/yelp/files"
print("catalog:", catalog, "| volume path:", VOL)
```

- Builds the **file path** to that Volume and prints it so you can see where files go.

Step 1 — Create the Yelp-style Parquet

```
business = [
    ("B001", "Sunrise Diner", "Phoenix", "AZ", "Restaurants", 4.0, 120),
    ...
]
```

- A plain **Python list of tuples** — 5 sample businesses. The columns that will *change over time* (to demo SCD) are **stars**, **city**, **review_count**.

```
biz_df = spark.createDataFrame(business,  
    ["business_id","name","city","state","category","stars","review_count"])
```

- `spark.createDataFrame(data, columns)` turns the Python list into a **distributed Spark DataFrame**, giving each tuple position a column name. Spark infers types (string, double, long).

```
biz_df.write.mode("overwrite").parquet(f"{VOL}/business")
```

- `.write` starts a write; `.mode("overwrite")` replaces any existing data (safe re-run); `.parquet(path)` saves it as **Parquet** files in the Volume. This is your "Yelp Parquet file."

```
display(dbutils.fs.ls(f"{VOL}/business"))
```

- `dbutils.fs.ls(path)` lists the files written (you'll see `part-*.snappy.parquet`). `display(...)` renders it as a table in the notebook.

Step 2 — Load the Parquet (initial dimension load)

```
biz = spark.read.parquet(f"{VOL}/business")
```

- `spark.read.parquet(path)` reads the Parquet back into a DataFrame. Parquet is self-describing, so the schema/types come back automatically.

```
biz.printSchema()  
display(biz)
```

- `printSchema()` prints columns + types; `display(biz)` shows the rows. This is your starting "current state" of the dimension.

Step 3 — Build the SCD1 dimension table (Delta)

```
(biz.write.format("delta").mode("overwrite").saveAsTable("yelp.dim_business_scd1"))
```

- `.format("delta")` writes a **Delta** table (Parquet + a transaction log → ACID + MERGE).
- `.saveAsTable("yelp.dim_business_scd1")` registers it as a **managed table** in the catalog (queryable by name), rather than just files. This is your SCD1 dimension (one row per business).

```
display(spark.table("yelp.dim_business_scd1").orderBy("business_id"))
```

- `spark.table(name)` reads a registered table; `.orderBy("business_id")` sorts for readability.

Step 4 — The change batch (the "next load")

```
changes = spark.createDataFrame([
    ("B001",...,4.0,120),    # unchanged
    ("B002",...,4.7,350),    # changed
    ("B003", "...", "Boulder",...,95), # changed city + reviews
    ("B006",...,4.1,60),     # NEW
], [...])
```

- A new DataFrame simulating the **next data load**. It deliberately mixes the four scenarios: **unchanged (B001)**, **changed (B002, B003)**, **new (B006)** — so you can see how SCD handles each.

Step 5 — SCD1 MERGE (overwrite in place) + validation

```
tgt = DeltaTable.forName(spark, "yelp.dim_business_scd1")
```

- Wraps the Delta table as a `DeltaTable` object so we can call `.merge(...)` on it.

```
(tgt.alias("t").merge(changes.alias("s"), "t.business_id = s.business_id")
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute())
```

- `.alias("t")` / `changes.alias("s")` — name the **target** and **source** so the condition can reference them.
- `.merge(source, condition)` — start an upsert; match rows where `business_id` is equal.
- `.whenMatchedUpdateAll()` — for existing businesses, **overwrite all columns** with the new values (this is SCD Type 1 — the old values are lost).
- `.whenNotMatchedInsertAll()` — for a business not already present (B006), **insert** it.
- `.execute()` — actually run the MERGE.

```
display(spark.sql("""
    SELECT COUNT(*) AS total_rows, COUNT(DISTINCT business_id) AS distinct_biz
    FROM yelp.dim_business_scd1"""))
```

- Validation: `*COUNT( )` must equal `COUNT(DISTINCT business_id)` → **proves there's exactly one row per business**** (no history/duplicates), which is the defining property of SCD1.

```
display(spark.sql("SELECT * FROM yelp.dim_business_scd1 WHERE business_id IN ('B002', 'B003', 'B006')"))
```

- Shows the updated/new rows — B002 = 4.7/350, B003 = Boulder, B006 exists. The **old values are gone** (no history).

Step 6 — Build the SCD2 dimension (with history columns)

```
scd2_init = (biz
    .withColumn("start_date", F.lit("2026-01-01").cast("date"))
    .withColumn("end_date", F.lit(None).cast("date"))
    .withColumn("is_current", F.lit(True))
    .withColumn("business_sk", F.sha2(F.concat_ws("||", F.col("business_id"), F.lit("2026-01-01")), 256)))
```

- `.withColumn(name, expr)` adds/replaces a column.
- `F.lit("2026-01-01").cast("date")` — `F.lit` makes a literal value; `.cast("date")` turns the string into a real DATE. `start_date` = when this version became effective.
- `end_date = F.lit(None).cast("date")` — NULL (open-ended) because the initial version is still current.
- `is_current = F.lit(True)` — a boolean flag marking the active version.
- `business_sk` — a **surrogate key**: `F.concat_ws("||", ...)` joins `business_id` + the date with a `||` separator, and `F.sha2(..., 256)` hashes it to a unique, stable key per *version* (so the same business can have multiple surrogate keys over time — essential for SCD2).

```
(scd2_init.write.format("delta").mode("overwrite").saveAsTable("yelp.dim_business_scd2"))
```

- Saves the SCD2 dimension as a Delta table (now with history columns).

Step 7 — Prepare the change batch with an effective date

```
EFF = "2026-07-01"
changes2 = changes.withColumn("effective_date", F.lit(EFF).cast("date"))
```

- Reuses the same `changes` DataFrame but stamps an `effective_date` of 2026-07-01 — the date the new versions take effect. (Real pipelines use the load/batch date.)
-

Step 8 — SCD2 MERGE: expire old + insert new (the mergeKey pattern)

Why it's tricky: a changed business needs **two** actions in one load — *update* the current row (to expire it) **and** *insert* a new row (the new version). A single MERGE can't update and insert the *same* matched row. The **mergeKey trick** solves this by feeding the source **twice**.

```
current = spark.table("yelp.dim_business_scd2").filter("is_current = true")
```

- The set of **currently-active** rows (one per business) we compare incoming data against.

```
changed = (changes2.alias("s")
    .join(current.alias("c"), "business_id")
    .where("s.stars <> c.stars OR s.city <> c.city OR s.review_count <> c.review_count")
    .select("s.*"))
```

- `.join(current, "business_id")` — match incoming rows to their current version.
- `.where("... <> ...")` — keep only rows where a **tracked attribute actually changed** (`<>` means "not equal"). Unchanged rows are excluded (no new version needed).
- `*.select("s.*")` — keep just the source columns. `changed` = "businesses that changed."

```
staged = (changed.withColumn("mergeKey", F.lit(None).cast("string"))
    .unionByName(changes2.withColumn("mergeKey", F.col("business_id"))))
```

- This builds the **doubled source**:
- `changed with mergeKey = NULL` — because NULL never equals the target key, these rows will **NOT match** → they become the **INSERTs** (the new versions).
- `changes2 with mergeKey = business_id` — the full batch keyed normally; these **WILL match** the current rows → used to **EXPIRE** them (and new business B006 won't match → also inserts as current).
- `.unionByName(...)` stacks the two sets by column name.

```
tgt2 = DeltaTable.forName(spark, "yelp.dim_business_scd2")
(tgt2.alias("t").merge(staged.alias("s"), "t.business_id = s.mergeKey AND t.is_current = true")
```

- MERGE condition matches on `business_id = mergeKey` and only against the current row (`is_current = true`).

```
.whenMatchedUpdate(
    condition="t.stars <> s.stars OR t.city <> s.city OR t.review_count <> s.review_count",
    set={"is_current": "false", "end_date": "s.effective_date"})
```

- For a **matched** current row whose attributes changed: **expire it** — set `is_current=false` and stamp `end_date` = the effective date. (The `condition` guards so *unchanged* businesses in the batch aren't touched.)

```
.whenNotMatchedInsert(values={
  "business_id":"s.business_id", ... ,
  "start_date":"s.effective_date","end_date":"NULL","is_current":"true",
  "business_sk":"sha2(concat_ws('||', s.business_id, s.effective_date), 256)}")
.execute()
```

- For **non-matching** source rows (the NULL-mergeKey new versions, and brand-new B006): **insert a new current version** — `start_date` = effective date, `end_date` = NULL, `is_current` = `true`, and a fresh surrogate key from the new effective date.
- `.execute()` runs it. Net effect per changed business: old row expired + new row inserted.

```
display(spark.table("yelp.dim_business_scd2").orderBy("business_id","start_date"))
```

- Shows the result — changed businesses now have **two rows** (old expired, new current).

Step 9 — Validate SCD2

```
-- 9a: history for B002
SELECT ... FROM yelp.dim_business_scd2 WHERE business_id='B002' ORDER BY start_date
```

- Two rows for B002: the **old version** (`is_current=false`, `end_date` set) and the **new version** (`is_current=true`, `end_date=NULL`). This is the proof SCD2 kept history.

```
-- 9b: one current row per business
SELECT business_id, COUNT(*) FROM ... WHERE is_current=true GROUP BY business_id
```

- Every count must be **1** — exactly one active version per business (no overlaps).

```
-- 9c: point-in-time
WHERE business_id='B003' AND start_date <= DATE'2026-03-01'
AND (end_date > DATE'2026-03-01' OR end_date IS NULL)
```

- Reconstructs history: "what was B003 on **2026-03-01**?" The filter selects the version whose validity window **contains** that date → returns **Denver** (the pre-change value). This is the killer feature of SCD2.

```
-- 9d: integrity
bad_multiple_current, b002_versions (=2), b001_versions (=1)
```

- **bad_multiple_current** must be 0 (no business has 2 current rows); **B002 has 2 versions** (changed), **B001 has 1** (unchanged). Confirms correctness.

Step 10 — Scenario recap (markdown)

The table summarizes how each source event is handled:

Event	SCD1	SCD2
New (B006)	insert	insert (current)
Changed (B002/B003)	overwrite (history lost)	expire old + insert new
Unchanged (B001)	no-op	no-op
Deleted at source	delete/flag	soft-delete: <code>is_current=false</code> + <code>end_date</code>

Bonus — soft delete for SCD2:

```
spark.sql("""UPDATE yelp.dim_business_scd2
            SET is_current=false, end_date=DATE'2026-07-01'
            WHERE business_id IN ('B004') AND is_current=true""")
```

- Marks a business no longer active **without deleting history** — you can still see it existed and when it ended.

The 5 ideas to remember

1. **Delta + MERGE** is what makes upserts/SCD possible on a lake.
2. **SCD1 = overwrite** (`whenMatchedUpdateAll`) → one row per key, no history.
3. **SCD2 = versioning** with `start_date` / `end_date` / `is_current` + a **surrogate key**.
4. The **mergeKey pattern** (feed the source twice, NULL key for inserts) does "expire + insert" in one MERGE; the two-step SQL (UPDATE then INSERT) is the easy-to-read alternative.
5. **Validate** with: one-row-per-key (SCD1), one-current-per-key + point-in-time (SCD2).